

Project: Nexus Server — HTTP Server in C++20

Document: Engineering Documentation & Architecture | Author: Orel Shushan

Table of Contents

- Chapter 1: Network Infrastructure and TCP Server Lifecycle
- Chapter 2: HTTP Request Parsing and Input Handling
- Chapter 3: Concurrency Engine and Thread Pool Management
- Chapter 4: Static File Serving & Security
- Chapter 5: Performance and Load Testing (k6)
- Engineering Executive Summary

1. Network Infrastructure and TCP Server Lifecycle

1.1 Overview of the HttpServer Class

Class Role: The class serves as the main entry point and primary management interface for the server. It is responsible for initializing the network infrastructure and sockets (setup_socket), managing the listening and client acceptance loop with support for graceful shutdown (start based on the is_running_ flag), and routing each client's handling (handle_client) to be executed on a separate thread via the ThreadPool.

:Class Fields and Properties

(Field Name (Type	Description
(port_ (uint16_t	The system-level port number on which the socket .listens for incoming HTTP requests
(server_fd_ (int	The File Descriptor representing the listening socket of the server facing the operating system. Its sole purpose is to listen for and accept connection requests (associated with the local address and port_, not a specific client). Initialized to -1 to signify that it has not .been created yet
(is_running_ (bool	A boolean flag indicating the server state. It is set to true when the server starts via start(), keeps the main listening loop active, and changes to false when .stop() is called for a graceful shutdown
thread_pool_ ((WorkerThreadPool	A thread pool object managing a task queue and assigning a dedicated worker thread for each incoming client. This enables concurrent processing of multiple clients simultaneously and prevents a single client from .blocking the entire server

1.2 What is a File Descriptor (FD)?

A **File Descriptor (FD)** is a non-negative integer (`int`) serving as an identifier and intermediary between User Space and the Operating System Kernel.

- **Kernel Interaction:** When the program creates a new resource (such as a communication socket or a file on disk), the kernel allocates and manages the complex data structure of that resource inside its kernel space memory (Kernel Space). Instead of exposing this internal structure, the kernel returns a numerical index from the process's resource table (File Descriptor Table) to the program.
- **Usage in Code:** In every subsequent operation performed by the server (listening, reading, writing, or closing the connection), the program queries the kernel by passing that same integer (`server_fd_` or `client_fd`), and the OS knows exactly which physical network connection to execute the operation on.

The "Everything is a file" Principle: In POSIX/Linux-based operating systems, all •
Input/Output (I/O) operations are handled uniformly via FDs, regardless of whether it
.is a local file on disk or a network communication socket

TCP Lifecycle System Calls 1.3

1.3.1 Creating the Endpoint — `socket()`

Calling this function makes a direct request to the kernel to allocate a new communication endpoint.

At this stage:

- The kernel allocates memory and internal data structures to manage the connection.
- The kernel returns a new `int` File Descriptor identifier.
- The socket is not yet bound to a specific IP address or port number.

Breakdown of Parameters for `socket(domain, type, protocol)`:

- **AF_INET (Address Family / Domain):**
 - *Essence:* Defines the address family and network protocol to be used for communication.
 - *In practice:* Represents IPv4 network addresses. Supporting IPv6 requires `AF_INET6`.
- **SOCK_STREAM (Socket Type):**
 - *Essence:* Defines the nature of communication and data transfer semantics at the Transport Layer.
 - *In practice:* Represents a reliable byte stream based on TCP, which includes:
 - **Connection-Oriented:** Requires a handshake and initial connection establishment prior to sending data.
 - **Reliability:** Every byte arrives at its destination without data loss and in exact sequential order.
- **0 (Protocol):**
 - *Essence:* Selects a specific protocol from the chosen protocol family.

In practice: Passing 0 instructs the kernel to automatically select the default protocol for the combination of Domain and Socket Type. For `AF_INET` and `SOCK_STREAM`, the default is always TCP (`IPPROTO_TCP`)

1.3.2 Binding Address and Port — bind()

The purpose of bind() is to associate (bind) the created socket (server_fd_) with a local IP address and a dedicated port on the host machine. Without this step, the kernel wouldn't know which process to route incoming TCP packets to when they arrive at the network interface card.

Parameters are configured using the sockaddr_in structure:

- **sin_family:** Set to AF_INET (IPv4 address family).
- **sin_addr.s_addr and INADDR_ANY:** The constant INADDR_ANY (equivalent to 0.0.0.0) instructs the kernel to listen on all available network interfaces on the machine (local network interfaces, Ethernet/Wi-Fi connections, and Localhost) rather than binding the server to a single IP address.
- **Byte Ordering and Conversions via htons and htonl:**
 - Modern processors with x86/ARM architecture store data in memory in **Little Endian** byte order (Least Significant Byte, LSB, saved at the lowest memory address).
 - On the other hand, network protocols (TCP/IP) require **Big Endian** byte order, defined as **Network Byte Order** (Most Significant Byte sent and stored first).
 - Conversion functions ensure compatibility with network hardware:
 - **htons()** (*Host to Network Short*): Converts the port number (16-bit variable).
 - **htonl()** (*Host to Network Long*): Converts the IP address (32-bit variable).

1.3.3 Transitioning to Passive Listening — listen()

By default, a created socket is an active socket intended to initiate outgoing communications. The call to `listen()` changes the state of the socket (`server_fd_`) from active to passive — from now on, it does not initiate connections, but waits for incoming connection requests from clients.

- **TCP 3-Way Handshake and Backlog Queue Structure:**

When a client contacts the server, a three-way handshake occurs at the OS kernel level:

1. The client sends a synchronization request (SYN).
2. The kernel responds with an acknowledgment and its own synchronization request (SYN-ACK).
3. The client acknowledges receipt (ACK).

Upon completion of the handshake in kernel space, the connection is ready and placed into the **Backlog Queue** — a managed waiting queue in kernel memory where the connection waits until the server retrieves it using `accept()`.

- **Configuring Queue Size via SOMAXCONN:**

The second parameter defines the capacity of the backlog queue. Choosing the system constant `SOMAXCONN` (Socket Maximum Connections) instructs the kernel to assign the maximum allowed queue size at the OS configuration level (instead of an arbitrarily small value). This setting is critical for handling high traffic loads (High Concurrency) and prevents connection rejections (Connection Refused) when a burst of fast requests arrives before being pulled from the queue.

1.3.4 Accepting an Incoming Connection — `accept()`

The `accept()` function is responsible for retrieving client connections that have completed the handshake with the OS.

- **Efficiency and CPU Burn Prevention (Blocking Mechanism):**
The function call is blocking. While the Backlog queue is empty of ready connections, the kernel puts the server's main thread to sleep instead of running an infinite loop that burns CPU resources (preventing busy-waiting). As soon as a new TCP connection enters the queue, the kernel immediately wakes up the process.
- **Dequeuing and Allocating `client_fd`:**
When a connection is available, `accept()` pops the first connection from the Backlog queue and returns a **brand-new File Descriptor (`client_fd`)** dedicated exclusively to communicating with that specific client, while `server_fd` continues listening normally for additional requests in the loop.

Storing Client Details and Graceful Exit: •

The function populates the `sockaddr_in client_addr` structure with the remote endpoint's IP address and port. If the server undergoes an intentional shutdown via `stop()`, the main socket is closed, `accept()` returns a negative value, and the `if (!is_running) break;` check allows a clean exit from the loop (Graceful Shutdown).

1.4 Separating Listening Socket and Connected Socket

The system maintains a complete architectural separation between the listening socket and client connection sockets:

- **Listening Socket (server_fd):**

A dedicated descriptor associated only with the local address and port. Its role is passive and limited to listening for and accepting new connection requests in the main server loop, without involvement in data transfer.

- **Connected Socket (client_fd):**

A unique descriptor created by the kernel upon calling `accept()`. This socket represents an established connection (Connected Socket) defined at the kernel level by a full TCP 4-tuple: local IP, local port, client IP, and client port. All exclusive communication with that client occurs through this socket (`recv` and `send`).

- **Importance of Separation and Preventing Task Contamination (Data Corruption):**

This separation is a fundamental requirement for the proper operation of a multi-threaded server:

1. **Service Continuity:** The server continues accepting new clients on the main thread without blocking while worker threads in the ThreadPools process data with existing clients.

Data Isolation: If all clients shared the same FD, different threads would read and write from the same channel, causing data corruption ("task contamination") and returning responses meant for one client to another client entirely.

1.5 Setting SO_REUSEADDR and TIME_WAIT State

Upon closing or restarting a TCP server, the OS places closed connections into a protocol-level hold state known as **TIME_WAIT**.

- **The Essence of TIME_WAIT at the Kernel Level:**

The OS does not release the port immediately, but keeps it reserved for a fixed duration (typically 1–2 minutes). This mechanism ensures that delayed or duplicate TCP segments wandering in the network dissipate completely and do not accidentally reach a new instance of the server and corrupt its data.

- **The Problem Without the Flag:**

If the server is restarted immediately after shutdown or crash, calling `bind()` will fail immediately and throw a system error `EADDRINUSE` (*Address already in use*), because the port is still marked as busy in the kernel.

Solution via SO_REUSEADDR: ●

Passing this flag via `setsockopt()` allows the server to bypass this restriction. The flag instructs the kernel to allow immediate rebinding of the local port, even if previous connections are currently in the `TIME_WAIT` state. This setting is essential for rapid restarts, high service availability, and development/testing environments where the server is brought up and down frequently.

Chapter 2: HTTP Request Parsing and Input Handling

2.1 HTTP/1.1 Protocol and Request Structure

An HTTP/1.1 request is defined as a structured text-based protocol consisting of three main parts:

- **Request Line:**
The first line of every request. It is divided into three components separated by single spaces:
 - **Method:** The action the client requests to perform (GET, POST, PUT, DELETE, etc.).
 - **Target / Path:** The requested resource path on the server (e.g., /, /index.html, /api/data).
 - **HTTP Version:** The protocol version used by the client (this server specifically targets HTTP/1.1 support).
- **Headers:**
Metadata lines appearing immediately after the request line in key-value pairs (Key : Value). Headers provide critical information about the request, such as the host identity (Host), data format (Content-Type), and exact body size in bytes (Content-Length).
- **Body / Payload:**
The actual content of the request (e.g., JSON object, plain text, or binary file content). Generally, GET requests omit the body (empty), whereas POST or PUT requests include payload data for server processing.

Role of CRLF and the Double CRLF Sequence:

- **Single CRLF (\r\n):** According to official HTTP/1.1 specs (RFC 9112), every line termination in the protocol must end with Carriage Return (\r / 0x0D) and Line Feed (\n / 0x0A).

Double CRLF Sequence (\r\n\r\n): The standard requires absolute separation between the header block and the message body using an empty line. This empty line is represented by two consecutive CRLF sequences (\r\n\r\n), acting as the delimiter that allows the parser to know precisely where headers end and the body begins.

2.2 Reading Data from Socket (recv) and Memory Management

The `recv()` function executes a system call to the kernel to pull the stream of bytes received from the client socket (`client_fd`) and copy them into the local memory buffer (`buffer`).

- **Meaning of Return Values of `recv()` (`bytes_read`):**
 - **`bytes_read > 0`:** The operation succeeded; the value represents the exact number of bytes fetched from the network and copied into the buffer.
 - **`bytes_read == 0`:** The client initiated a graceful connection closure (Graceful Close / TCP FIN / EOF). In this state, no more data remains to be read and the socket should be released.
 - **`bytes_read < 0`:** A network-level error or system call failure occurred, requiring immediate socket closure.
- **Memory Allocation Considerations and Buffer Structure:**
 - **4KB Buffer Size (`char buffer[4096]`):** Chosen to align with standard OS page size (4096 bytes). A 4KB capacity allows efficient reading of most standard HTTP/1.1 requests (including all headers) in a single I/O burst, avoiding expensive dynamic Heap allocations.
- **Preventing Buffer Overflow and Null Termination (`\0`):**
 - **Limiting Read to `sizeof(buffer) - 1`:** TCP transfers raw binary bytes without appending string terminators. Subtracting 1 byte ensures at least one free byte remains at the end of the array.

Setting `buffer[bytes_read] = '\0'`: Manually terminates the read buffer as a standard C-style string (Null-terminated String). This is critical before converting to `std::string`: without `\0`, the string constructor would read uninitialized memory beyond array bounds (Buffer Over-read), leading to undefined behavior, program crash, or sensitive memory exposure.

2.3 Request Parsing via HttpRequestParser

The `HttpRequestParser` class converts raw HTTP strings (`raw_request`) into an object-oriented `HttpRequest` data structure. The process runs sequentially using `std::istringstream` to treat the string as an input stream:

- **Extracting the Request Line:**
 - `std::getline(stream, line)` extracts the first line from the stream.
 - The line is passed to a local stream (`line_stream`) and extracted via stream extraction operators (`>>`):
 - First word -> `method (req.method)`.
 - Second word -> `path (req.path)`.
 - Third word -> `version (req.version)`.
- **Extracting Headers and Key/Value Structure:**
 - A `while (std::getline(stream, line))` loop processes header lines.
 - **Header End Detection:** Checked via `line.find(':')`. Upon reaching the blank separator line between headers and body, no colon is found, returning `std::string::npos`, breaking the loop.
 - **Parsing Lines:** Split into key (`line.substr(0, pos)`) and value (`line.substr(pos + 2)`).
 - Key is sliced from beginning up to colon.
 - Value is sliced starting 2 characters past colon to skip leading space.
 - **Cleaning Control Characters (\r):** `getline` splits on `\n`, leaving `\r`. `value.pop_back()` removes it.
 - Stored in an associative map (`req.headers[key] = value`) for fast header lookup.
- **Extracting Body:**
 - All remaining data in the input stream after the header loop breaks belongs to the message body.
 - A subsequent loop collects remaining lines into `req.body`

Chapter 3: Concurrency Engine and Thread Pool Management

3.1 ThreadPool Architecture and Data Structures

In high-load network systems, a pre-allocated thread pool model is strictly preferred over allocating a dedicated thread per request (Thread-per-Client):

- **Preventing Resource Starvation and System Crashes:**
Opening a thread per client under traffic spikes (thousands of simultaneous requests) allocates thousands of threads, consuming system memory (each thread requiring a multi-megabyte call stack), leading to connection rejections and process crashes.
- **Reducing Context Switching Overhead:**
Frequent thread creation and destruction require costly system calls. Thousands of active threads cause heavy Context Switching on the CPU. The ThreadPool maintains a fixed number of worker threads matching processing capacity, reused continuously to handle queued tasks.

Class Variables and Their Roles:

- **workers_ (std::vector<std::thread>):**
Stores and manages server worker threads, initialized in the constructor and executing worker_loop in the background.
- **tasks_ (std::queue<std::function<void()>>):**
FIFO task queue holding abstract function wrappers (std::function<void()>) awaiting execution by an available worker thread.
- **queue_mutex_ (std::mutex):**
Mutual exclusion lock protecting the task queue (tasks_) against simultaneous concurrent access, preventing race conditions.
- **cv_ (std::condition_variable):**
Condition variable used for signaling between the main task-producing thread and worker threads. Allows idle threads to sleep efficiently when the queue is empty and wake up upon task arrival without wasting CPU cycles.

stop_ (bool): •

Boolean control flag indicating whether the pool is shutting down, allowing threads to .exit gracefully

3.2 Synchronizing Task Queue and Enqueuing Tasks (enqueue)

The task queue (`tasks_`) is a critical shared resource: the main thread pushes tasks via `push`, while worker threads pop tasks via `pop`.

- **Race Condition Mitigation:**
Unsynchronized access corrupts internal pointers and size counters. `std::mutex` provides mutual exclusion, allowing only one thread to read or update the queue at any given moment.
- **RAII Lock Management with Scoped Blocks (`{}`):**
Uses `std::unique_lock`:
 - Lock acquired on construction and automatically released when exiting the scoped block `{}`.
 - **Exception Safety:** Ensures locks release properly if exceptions occur.
 - **Minimal Critical Section:** Limits lock duration to `tasks_.push`, releasing resources quickly.
- **Efficient Signaling and Preventing Thundering Herd:**
 - `cv_.notify_one()` is called outside the lock block, preventing "Hurry-up and Wait" phenomena.
 - Wakes up exactly one sleeping thread per enqueued task, avoiding unnecessary context switches

3.3 Worker Thread Loop (`worker_loop`) and Waiting Management

Each worker thread runs `worker_loop` continuously, synchronizing sleeping state with task retrieval:

- **Efficient Waiting with `cv_.wait()` (Avoiding Busy-Waiting):**
Calling `cv_.wait(lock, ...)` atomically releases the lock and puts the thread to sleep in the kernel. Upon receiving a notification signal, it re-acquires the lock automatically before proceeding.

- **Predicate Lambda and Spurious Wakeups:**
Protects against system-level spurious wakeups via `[this]() { return stop_ || !tasks_.empty(); }`.
- **Termination Condition and Graceful Shutdown:**
`if (tasks_.empty() && stop_) return;` exits cleanly when stopping and no pending tasks remain.
- **Separating Task Retrieval from Execution:**
Task retrieval happens under lock protection, but the actual task execution runs entirely outside the lock, enabling true parallelism.
 - This ensures that handling a client's HTTP request does not block other worker threads from acquiring new tasks from the queue.
 - Maximizes parallelism and maintains high server concurrency.
- **Destruction and Cleanup:**
- Allows worker threads to exit their loops smoothly without abruptly dropping connections.
 - Prevents memory leaks and undefined behavior upon destruction.
- Ensures all pooled resources are released back to system control safely ○

3.4 Destruction and Graceful Shutdown (~WorkerThreadPool)

The destructor shuts down resources gracefully:

- `stop_` set to true under mutex protection for memory visibility.
- `cv_.notify_all()` wakes up all sleeping threads.
- `worker.join()` blocks the main thread until each worker finishes cleanly, preventing `std::terminate` or abrupt connection cuts.
- Guarantees all pending tasks in execution are given adequate time to complete
- `.safely`

Chapter 4: Static File Serving & Security

4.1 Mapping URLs to Local File System

Translates virtual HTTP request paths (`req.path`) to physical disk files under a dedicated root directory (e.g., `public/`).

- Concatenates root directory with request path (e.g., `/assets/style.css` -> `./public/assets/style.css`).
- `./` Serves default index document (`index.html`) when accessing root

4.2 Binary File Reading and MIME Types

Binary Reading (`std::ios::binary`): Prevents line-ending conversions (`\r\n` vs `\n`) that corrupt images, scripts, or WASM binaries.

MIME Types (Content-Type): Ensures browser compatibility and prevents MIME-sniffing blocking. Defaults to `application/octet-stream` ○

4.3 Path Traversal Security Protection

Attack Vector: Prevents attackers from using `..` sequences (e.g., `GET ../../../../etc/passwd`) to escape the `public/` directory.

Defense Mechanism: Checks `req.path.find("..") != std::string::npos` before accessing disk. Immediately blocks suspicious requests with 403 Forbidden or 404 Not Found ○

Chapter 5: Performance and Load Testing (k6)

5.1 Methodology and Key Performance Indicators (KPIs)

Tested using **k6** with 100 Virtual Users (VUs) for 2 minutes.

Evaluated Throughput (RPS), Latency Profile (Average, Median, p95), and ○
.(Success Rate (target: 0% errors

5.2 Test Scenario and Actual Results

The server was tested under continuous load conditions locally for 2 consecutive minutes
:(with 100 concurrent Virtual Users (VUs

Metric	Actual Result	Engineering Significance
Total Requests Processed	+62,737 requests	Sustained traffic volume over 2 minutes without performance degradation
Success Rate	100.00%	dropped connections, 0 I/O 0 failures, 0 server errors
Throughput	RPS 522.4~	Stable processing rate exceeding 520 requests/sec
Average Latency	ms 6.86	Fast response times including disk file reads and network delivery
Median Latency	ms 5.29	Half of all requests answered in ~5 ms
p95 Latency	ms 17.24	Consistent performance even at upper load bounds

Engineering Executive Summary

In this project, I designed and built a zero-dependency, high-performance HTTP/1.1 server from scratch in Linux/POSIX using modern C++20. The primary goal was achieving direct mastery over low-level operating system and networking primitives: File Descriptor management, byte-level memory manipulations, multi-threaded synchronization, and robust security practices

System Layer Integration:

- **Network & POSIX Layer:**

Implemented TCP socket lifecycle (socket, setsockopt, bind, listen, accept), separating listening and connection sockets, with SO_REUSEADDR enabled.

- **Parsing & I/O Layer:**

Engineered a 4KB buffer (recv), null termination (\0), and HttpRequestParser compliant with RFC 9112 (Request Line, Headers, double CRLF \r\n\r\n boundary separation).

- **Concurrency & ThreadPool Layer:**

Designed a Producer-Consumer task queue with worker threads (WorkerThreadPool), RAII locking (std::unique_lock), condition variable signaling (notify_one), spurious wakeup prevention, and graceful shutdown

- **Static File Serving & Hardening:**

Implemented binary reading (std::ios::binary), MIME type mapping, and strict (path traversal protection (.. checking

Performance Validation:

- **Validated 62,700+ requests** processed over 2 minutes with **100 concurrent VUs**.
- **Maintained 100.00% success rate** (0 dropped connections, no deadlocks/leaks).

Achieved 522+ RPS, 5.29 ms median latency, and 17.24 ms p95 latency